simonwillison.net

Your job is to deliver code you have proven to work

Simon Willison

5–6 minutes

18th December 2025

In all of the debates about the value of AI-assistance in software development there's one depressing anecdote that I keep on seeing: the junior engineer, empowered by some class of LLM tool, who deposits giant, untested PRs on their coworkers—or open source maintainers—and expects the "code review" process to handle the rest.

This is rude, a waste of other people's time, and is honestly a dereliction of duty as a software developer.

Your job is to deliver code you have proven to work.

As software engineers we don't just crank out code—in fact these days you could argue that's what the LLMs are for. We need to deliver *code that works*—and we need to include *proof* that it works as well. Not doing that directly shifts the burden of the actual work to whoever is expected to review our code.

How to prove it works

There are two steps to proving a piece of code works.

Neither is optional.

The first is **manual testing**. If you haven't seen the code do the right thing yourself, that code doesn't work. If it does turn out to work, that's honestly just pure chance.

Manual testing skills are genuine skills that you need to develop. You need to be able to get the system into an initial state that demonstrates your change, then exercise the change, then check and demonstrate that it has the desired effect.

If possible I like to reduce these steps to a sequence of terminal commands which I can paste, along with their output, into a comment in the code review. Here's a recent example.

Some changes are harder to demonstrate. It's still your job to demonstrate them! Record a screen capture video and add that to the PR. Show your reviewers that the change you made actually works.

Once you've tested the happy path where everything works you can start trying the edge cases. Manual testing is a skill, and finding the things that break is the next level of that skill that helps define a senior engineer.

The second step in proving a change works is **automated testing**. This is so much easier now that we have LLM tooling, which means there's no excuse at all for skipping this step.

Your contribution should bundle the change with an automated test that proves the change works. That test should fail if you revert the implementation.

The process for writing a test mirrors that of manual testing:

get the system into an initial known state, exercise the change, assert that it worked correctly. Integrating a test harness to productively facilitate this is another key skill worth investing in.

Don't be tempted to skip the manual test because you think the automated test has you covered already! Almost every time I've done this myself I've quickly regretted it.

Make your coding agent prove it first

The most important trend in LLMs in 2025 has been the explosive growth of **coding agents**—tools like Claude Code and Codex CLI that can actively execute the code they are working on to check that it works and further iterate on any problems.

To master these tools you need to learn how to get them to *prove their changes work* as well.

This looks exactly the same as the process I described above: they need to be able to manually test their changes as they work, and they need to be able to build automated tests that guarantee the change will continue to work in the future.

Since they're robots, automated tests and manual tests are effectively the same thing.

They do feel a little different though. When I'm working on CLI tools I'll usually teach Claude Code how to run them itself so it can do one-off tests, even though the eventual automated tests will use a system like Click's CLIRunner.

When working on CSS changes I'll often encourage my coding agent to take screenshots when it needs to check if

the change it made had the desired effect.

The good news about automated tests is that coding agents need very little encouragement to write them. If your project has tests already most agents will extend that test suite without you even telling them to do so. They'll also reuse patterns from existing tests, so keeping your test code well organized and populated with patterns you like is a great way to help your agent build testing code to your taste.

Developing good taste in testing code is another of those skills that differentiates a senior engineer.

The human provides the accountability

A computer can never be held accountable. That's your job as the human in the loop.

Almost anyone can prompt an LLM to generate a thousand-line patch and submit it for code review. That's no longer valuable. What's valuable is contributing *code that is proven to work*.

Next time you submit a PR, make sure you've included your evidence that it works as it should.